CENG-322 Mobile Application Development - Deliverable 4

Team Name: IoT Innovators

Project Name: Smart Farm

Team Number: 8

Student's names and IDs:

- Chau Vi Cuong Nguyen – N01634611
- Hoang Tran – N01613506
- Tuan Anh Pham – N01238459
- Truong Anh Nguyen – N01639800

Table of Contents

Member Info and Participation 3

Project Description 3

What issue will your product solve. 3

Compare your application with existing apps in the market 3

GitHub Repo link 6

Coding work progress since deliverable 3. 6

List number of commits 6

Sprint goals 6

Sprint dashboard 7

Sprint Retrospective 10

C4 Model: System Context Diagram 11

C4 Model: Container Diagram 12

Design Patterns used in the code 13

Design Pattern 1: Model-View-Controller (MVC) Pattern 13

Design Pattern 2: Singleton Pattern 14

Unit Test Cases 15

Feedback Screen 16

Take screenshot showing all sensor data fetched and 23

updated from the DB. 23

Describe the main functionality added in this sprint 24

1. Real-Time Monitoring (Core Feature) 24

2. Device Control 24

3. Alerts and Notifications 24

4. Data History 24

5. User Authentication 24

Member Info and Participation

Name	Student ID	GitHub ID	Signature	Effort
Chau Vi Cuong Nguyen	N01634611	ChauViCuongNguyen4611	CVCN	100
Hoang Tran	N01613506	HoangTranTran3506	HT	100
Tuan Anh Pham	N01238459	TuanAnhPham8459	TA	100
Truong An Nguyen	N01639800	TruongAnNguyen9800	TAN	95

Project Description

The Smart Farm IoT application aims to create a comprehensive mobile solution for modern farming management. The final vision is to provide farmers with real-time monitoring and control of IoT-enabled farm equipment, including temperature monitors, automated irrigation systems, and crop health trackers.

What issue will your product solve.

The Problem:

- Newborn plants are highly sensitive to environmental conditions. Poor conditions can lead to illness, reduced productivity, and financial losses for farmers. Many small farms still rely on manual checking, which is uncentralized, time-consuming, and less accurate.

Our Solution: Smart Farm provides:

- Real-time monitoring of temperature, humidity, soil moisture, and light
- Instant alerts when conditions exceed safe thresholds
- Remote access through an Android app
- Affordable, easy-to-use system for small to medium-scale farmers

Compare your application with existing apps in the market

App 1: Farm Logs - https://www.farmlogs.com/

Farm Logs (now known as Bushel Farm) is a cloud-based farm management platform that leverages satellite imagery and data analytics. Its primary goal is to help farmers track field productivity, manage inputs, and gain operational insights.

Feature	Details
Main use	Farm management, record-keeping, and performance analysis.
Hardware	Software-only (no sensor included)
Cost	Subscription- based

App 2: CropX - https://www.cropx.com/

Feature	Details
Main use	Precision irrigation, soil monitoring, and agronomic recommendations.
Hardware	Proprietary multi-depth wireless soil sensors and telemetry gateways
Cost	High initial investment (~$1,500 per sensor kit) plus an annual subscription for the platform (~$399/year)

Our app: Smart Farm

Feature	Details
Main use	Real-time monitoring, instant alerts, and remote device control.
Hardware	Low-cost, open-source ESP32 sensors.
Cost	Free – No subscription fees (planned freemium in future).

Differences between our app and these two Apps

Feature	SmartFarm (Our App)	FarmLogs	CropX
Target User	Small to medium-scale farmers	Commercial/large farms	Commercial farms
Core Focus	Real-time monitoring + device control	Farm accounting + operations	Irrigation + agronomy
Hardware	Low-cost ESP32 (DIY)	None	Proprietary, premium sensors (~$1,500+)
Cost	Free	$19–149/month	$1,500+ sensor + $399/year

Device Control	Yes (ON/OFF)	No	Limited
Offline Support	Yes	Yes	Limited

Why do you believe your app is better than these two apps.

Our app offers the best balance of features and cost.

Accessible Cost: Smart Farm is completely free, eliminating the steep financial barriers presented by the subscription models of Farm Logs and the high upfront hardware costs of CropX.

Control & Simplicity: Unlike the software-only approach of Farm Logs, we provide affordable hardware integration. Unlike the complex, high-cost system of CropX, we empower farmers with direct, remote ON/OFF control of their devices using user-friendly, open-source technology.

Pros and cons of the three different apps.

	SmartFarm (Our App)	**FarmLogs**	**CropX**
Pros	Free to use Low-cost ESP32 hardware Real-time sensor data Remote device control French/English support	Comprehensive farm management Web + Mobile support Powerful analytics & satellite imagery No hardware required	Advanced agronomic models High-precision sensors Water & resource optimization
Cons	Android only Requires hardware setup No web interface yet	Subscription fees ($19-149/mo) No hardware integration Costly for small farms	Expensive hardware ($1,500+) Complex setup

GitHub Repo link

https://github.com/ChauViCuongNguyen4611/SmartFarm

Coding work progress since deliverable 3.

Name	Sprint 4 Work Completed
Chau Vi Cuong Nguyen	- Unit tests (10+); - Add function for alert screen - Add function to account fragment (change password, update username)
Hoang Tran	- Account Fragment UI (profile card, editable fields, non-editable email); - Change Password dialog UI; - Update alert screen UI
Tuan Anh Pham	- Sensor data fetch/update from Firebase - Update analytics & graph screen to show real time information
Truong An Nguyen	- Continue working on feedback screen - Fixed tool bar to meet the requirement

List number of commits

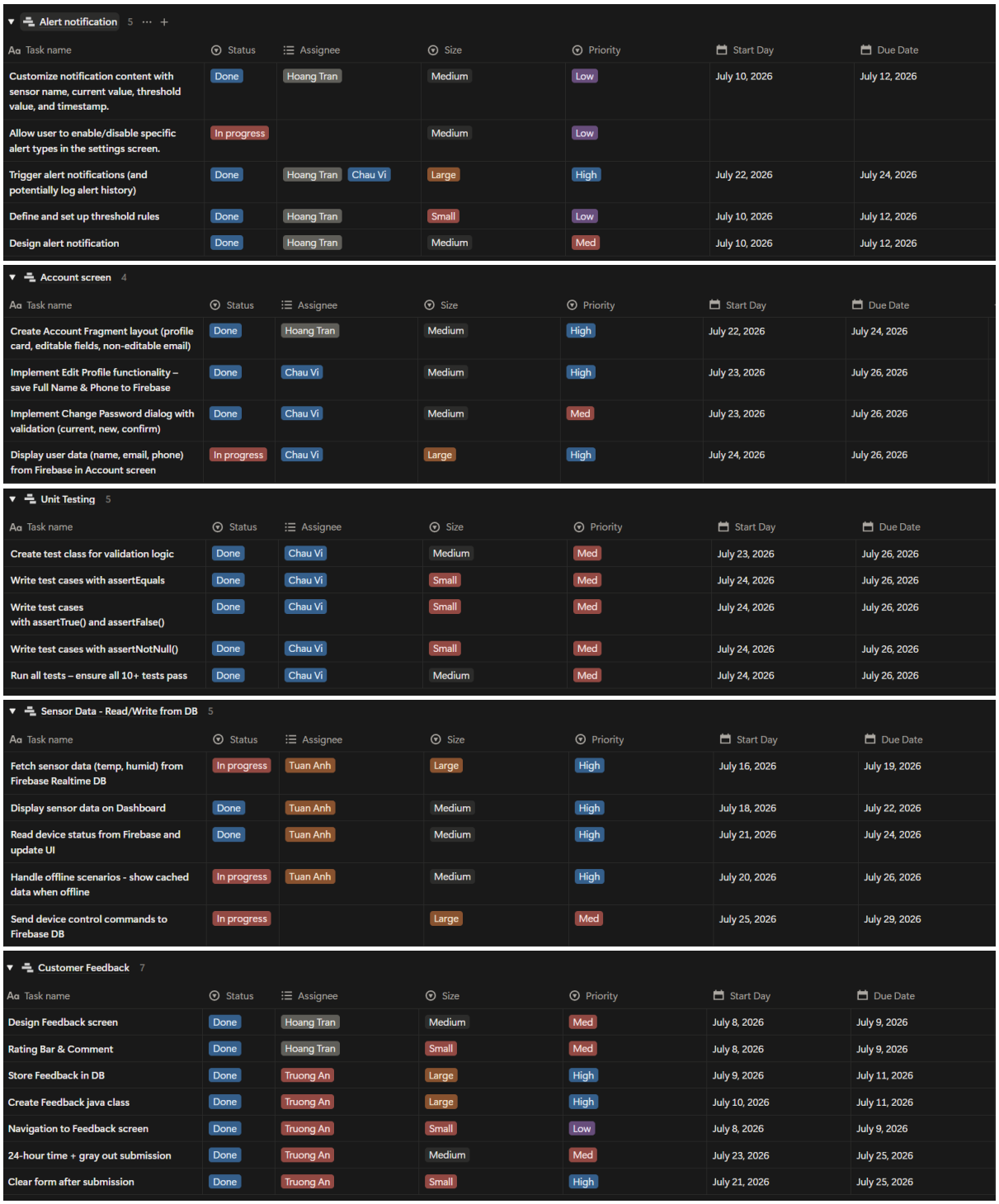
Team member	Chau Vi Cuong Nguyen	Hoang Tran	Tuan Anh Pham	Truong An Nguyen
July 20	2			2
July 21	4			1
July 22	1	5	5	2
July 23		3	5	
July 24	5	2		5

Sprint goals

1. Complete Feedback Screen with ProgressBar, AlertDialog, and 24-hour timer
2. Implement Regex validation for Login and Registration screens
3. Write 10+ unit test cases with assert methods – all passing
4. Fetch and display sensor data from Firebase in real-time
5. Continue working on alert screen

6. Start working on account screen

Sprint dashboard



Alert notification 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Customize notification content with sensor name, current value, threshold value, and timestamp.	Done	Hoang Tran	Medium	Low	July 10, 2026	July 12, 2026
Allow user to enable/disable specific alert types in the settings screen.	In progress		Medium	Low		
Trigger alert notifications (and potentially log alert history)	Done	Hoang Tran Chau Vi	Large	High	July 22, 2026	July 24, 2026
Define and set up threshold rules	Done	Hoang Tran	Small	Low	July 10, 2026	July 12, 2026
Design alert notification	Done	Hoang Tran	Medium	Med	July 10, 2026	July 12, 2026

Account screen 4

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Create Account Fragment layout (profile card, editable fields, non-editable email)	Done	Hoang Tran	Medium	High	July 22, 2026	July 24, 2026
Implement Edit Profile functionality – save Full Name & Phone to Firebase	Done	Chau Vi	Medium	High	July 23, 2026	July 26, 2026
Implement Change Password dialog with validation (current, new, confirm)	Done	Chau Vi	Medium	Med	July 23, 2026	July 26, 2026
Display user data (name, email, phone) from Firebase in Account screen	In progress	Chau Vi	Large	High	July 24, 2026	July 26, 2026

Unit Testing 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Create test class for validation logic	Done	Chau Vi	Medium	Med	July 23, 2026	July 26, 2026
Write test cases with assertEquals	Done	Chau Vi	Small	Med	July 24, 2026	July 26, 2026
Write test cases with assertTrue() and assertFalse()	Done	Chau Vi	Small	Med	July 24, 2026	July 26, 2026
Write test cases with assertNotNull()	Done	Chau Vi	Small	Med	July 24, 2026	July 26, 2026
Run all tests – ensure all 10+ tests pass	Done	Chau Vi	Medium	Med	July 24, 2026	July 26, 2026

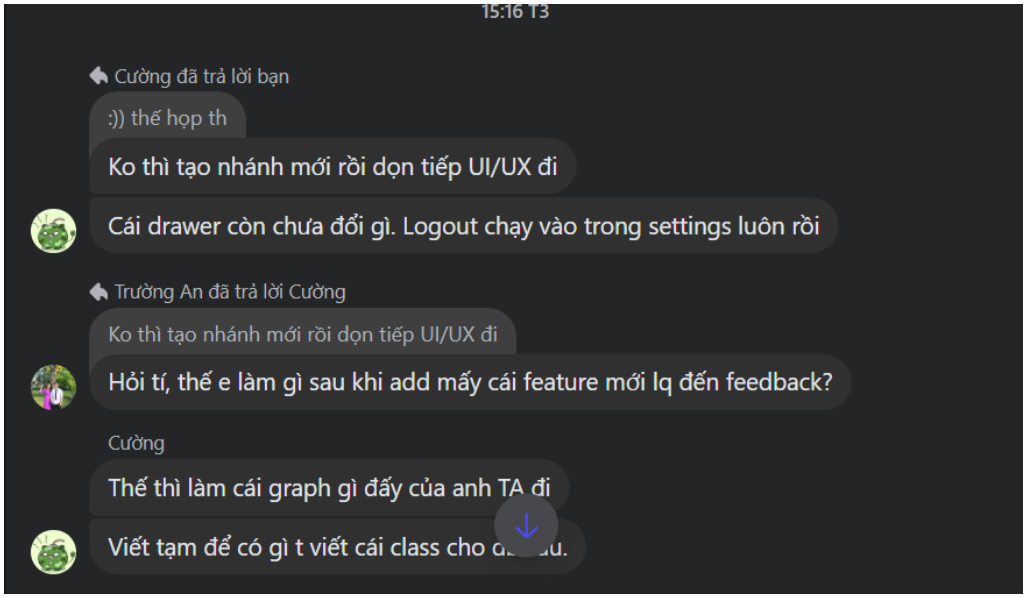
Sensor Data - Read/Write from DB 5

Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Fetch sensor data (temp, humid) from Firebase Realtime DB	In progress	Tuan Anh	Large	High	July 16, 2026	July 19, 2026
Display sensor data on Dashboard	Done	Tuan Anh	Medium	High	July 18, 2026	July 22, 2026
Read device status from Firebase and update UI	Done	Tuan Anh	Medium	High	July 21, 2026	July 24, 2026
Handle offline scenarios - show cached data when offline	In progress	Tuan Anh	Medium	High	July 20, 2026	July 26, 2026
Send device control commands to Firebase DB	In progress		Large	Med	July 25, 2026	July 29, 2026

Customer Feedback 7

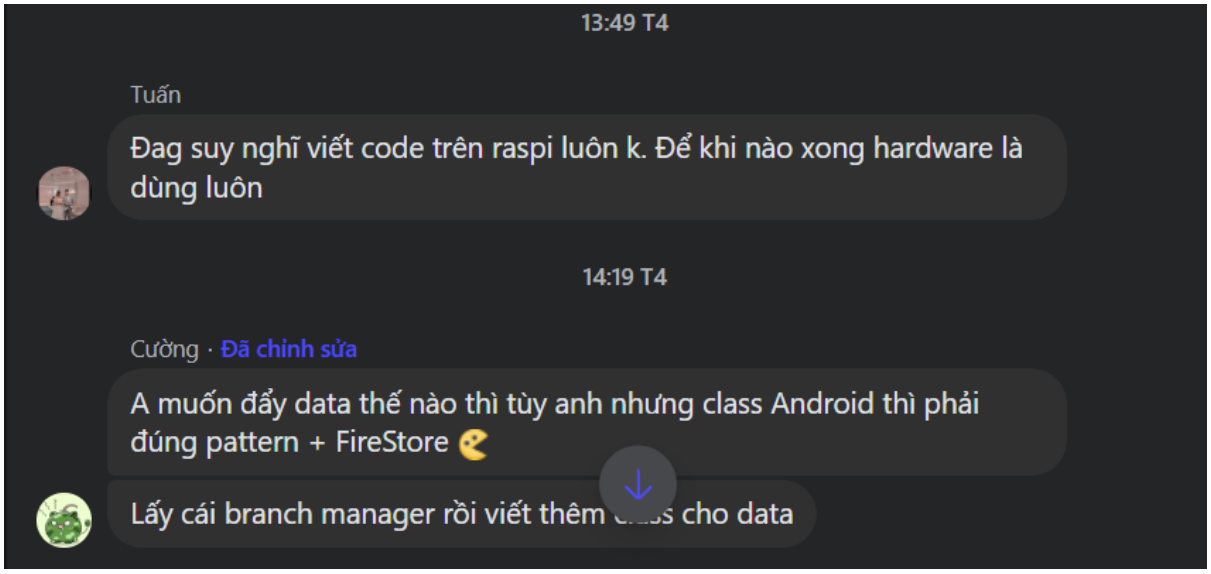
Task name	Status	Assignee	Size	Priority	Start Day	Due Date
Design Feedback screen	Done	Hoang Tran	Medium	Med	July 8, 2026	July 9, 2026
Rating Bar & Comment	Done	Hoang Tran	Small	Med	July 8, 2026	July 9, 2026
Store Feedback in DB	Done	Truong An	Large	High	July 9, 2026	July 11, 2026
Create Feedback java class	Done	Truong An	Large	High	July 10, 2026	July 11, 2026
Navigation to Feedback screen	Done	Truong An	Small	Low	July 8, 2026	July 9, 2026
24-hour time + gray out submission	Done	Truong An	Medium	Med	July 23, 2026	July 25, 2026
Clear form after submission	Done	Truong An	Small	High	July 21, 2026	July 25, 2026

Take screenshots showing the team discussion, topics discussed, team members.



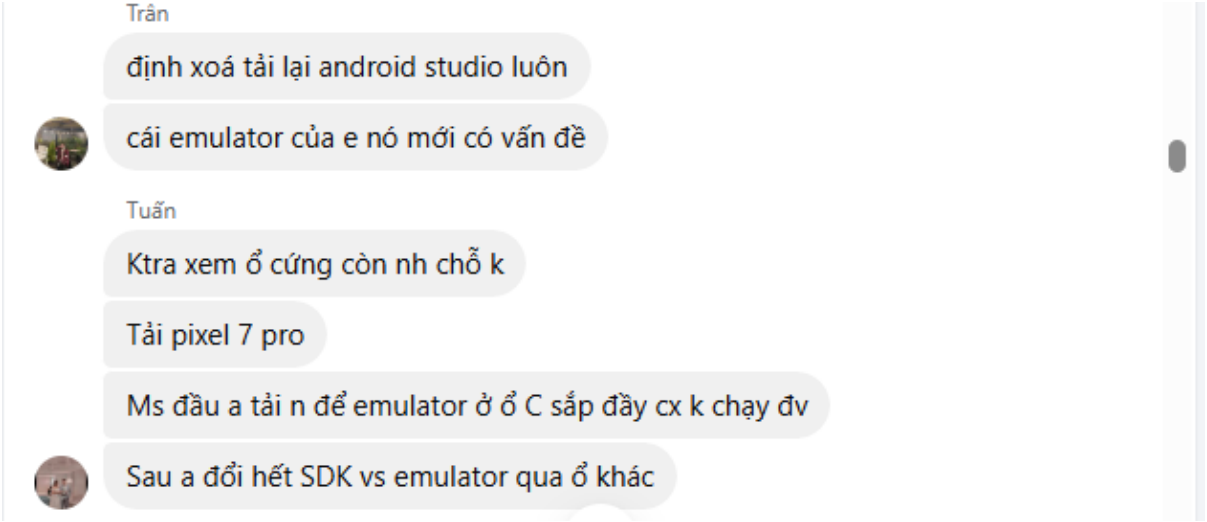
Date: Tuesday, 7/21/2026

What we discuss: Cleaning the UI/UX design



Date: Wednesday, 7/22/2026

What we discuss: How we connect the data from raspberryPi to the overall database

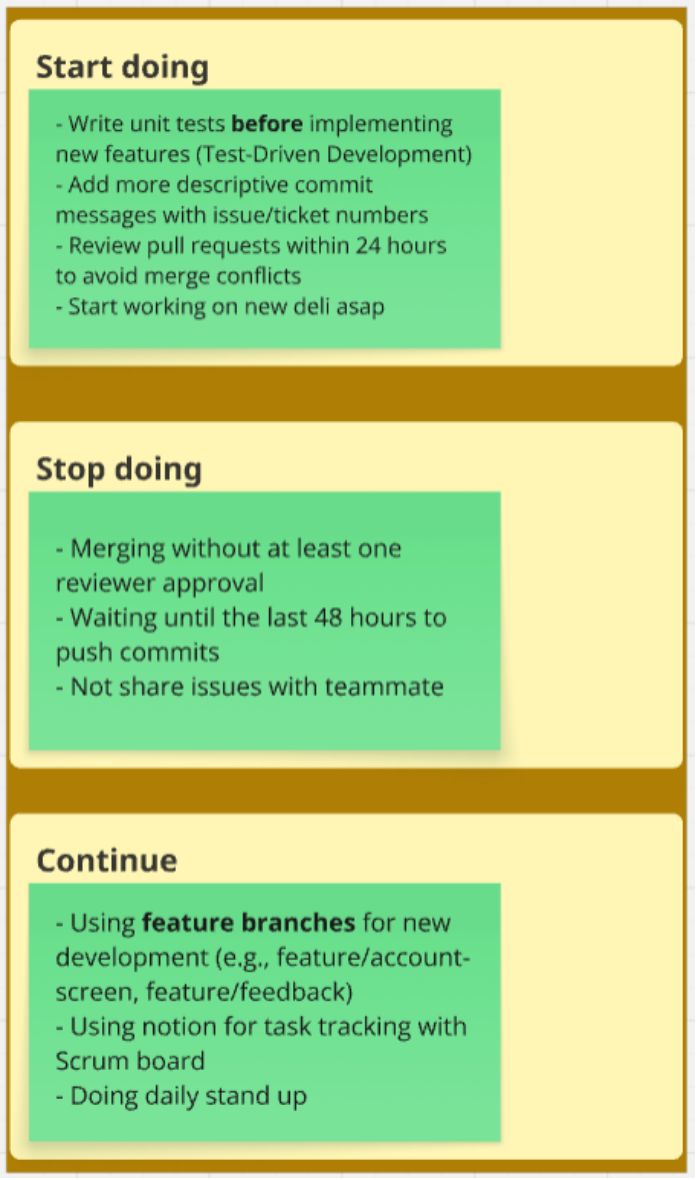


Date: Thursday, 7/23/2026

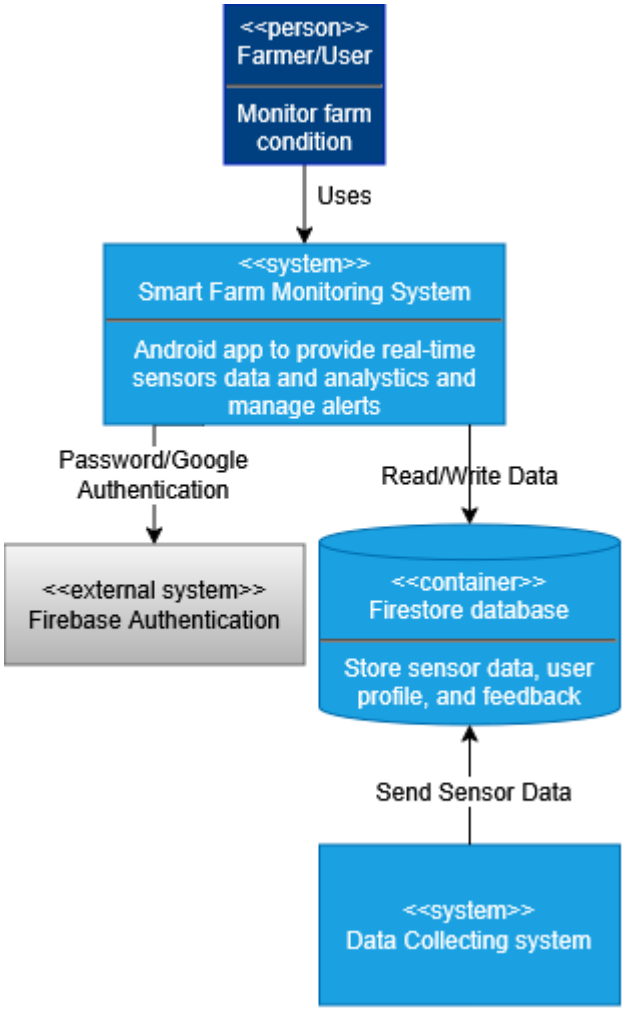
What we discuss: How to fix problem releated to android studio not working properly

Sprint Retrospective

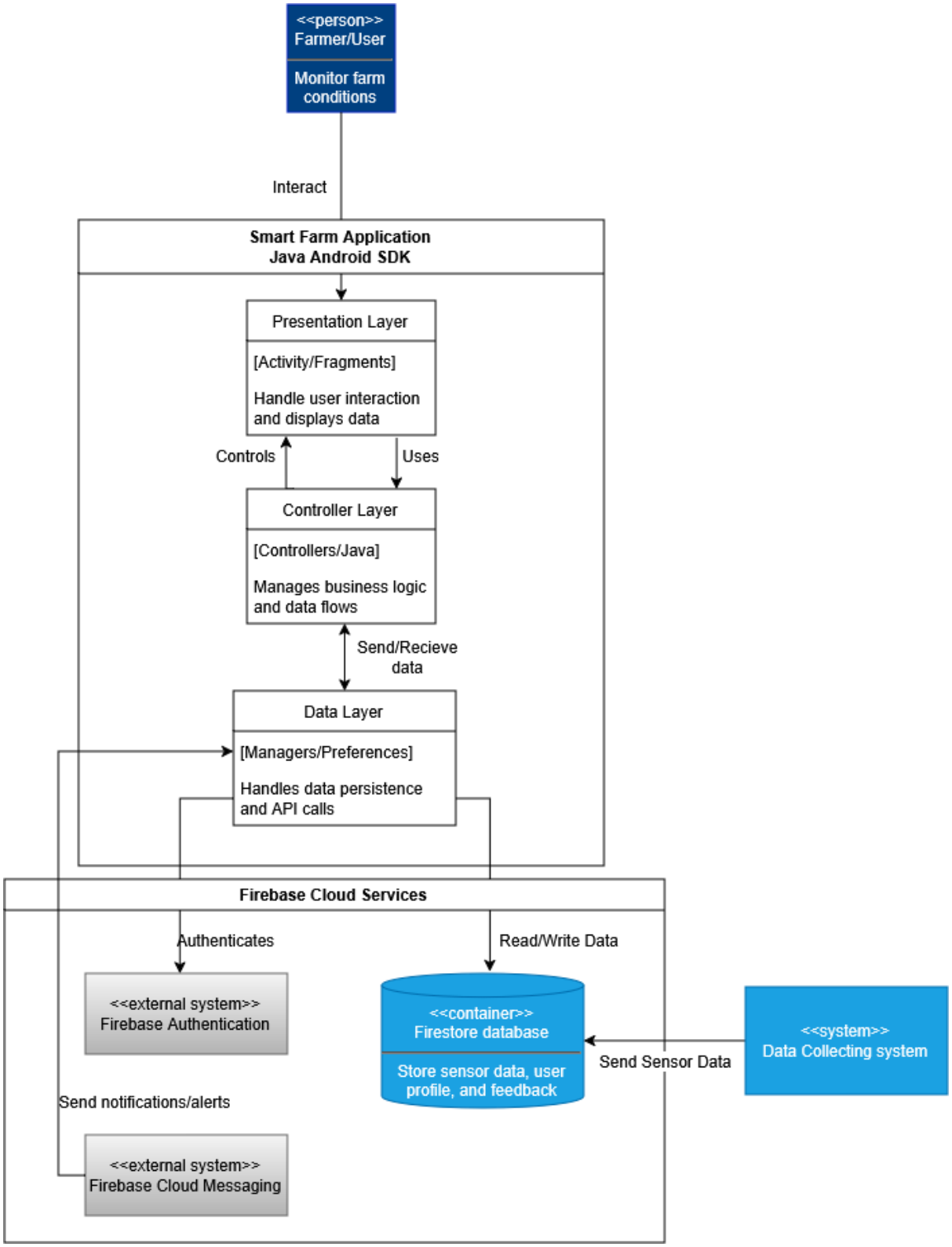
Tool Used: Micro
Date: July 26, 2026
Sprint: 4 (July 13 – July 26, 2026)
Attendees: Chau Vi Cuong Nguyen, Hoang Tran, Tuan Anh Pham, Truong An Nguyen
Missed the Meeting: None (all members present)



C4 Model: System Context Diagram



C4 Model: Container Diagram



Design Patterns used in the code

Design Pattern 1: Model-View-Controller (MVC) Pattern

1. Model – Data and Business Rules

```
// UserModel.java - Data Model
public class UserModel {
    private String userId;
    private String email;
    private String fullName;
    private String phone;
    private long createdAt;
    // Getters and setters...
}

// InputChecker.java - Business Rules
public class InputChecker {
    public ValidationResult validateSignup(String email, String password,
                                           String fullName, String phone,
                                           String confirmPassword) {
        // Validation logic...
    }
}

```

2. View – UI Presentation

```
// SignupActivity.java - View
public class SignupActivity extends BaseActivity
        implements SignupController.SignupView {

    private TextInputEditText etEmail, etPassword;
    private MaterialButton btnSignup;

    @Override
    public void showEmailError(String err) {
        tilEmail.setError(err);  // UI update
    }

    @Override
    public void onSignupSuccess() {
        Toast.makeText(this, "Success", Toast.LENGTH_SHORT).show();
        navigateToMain();  // Navigation
    }
}

```

3. Controller – Mediates Between Model and View

```
// SignupController.java - Controller
public class SignupController implements FirebaseAuthManager.AuthCallback {
    private final SignupView view;
    private final InputChecker inputChecker;
    private final FirebaseAuthManager authManager;

    public void handleSignup(String email, String password, String fullName,
                             String phone, String confirmPassword, boolean termsAccepted) {
        // Business logic
        InputChecker.ValidationResult result =
            inputChecker.validateSignup(email, password, fullName, phone, confirmPassword);

        if (!result.isValid()) {
            displayValidationErrors(result);  // Update view
            return;
        }
```

```
17.
18.         view.showLoading(true);  // Update view
19.         authManager.signUp(email, password);  // Call model
20.     }
21.
22.     @Override
23.     public void onSuccess(FirebaseUser user) {
24.         // Handle successful authentication
25.         UserModel userModel = new UserModel(user.getUid(), user.getEmail(),
26.                                             signupFullName, signupPhone);
27.         // Save to database...
28.     }
29. }
30.
```

Explanation:

- Separation of Concerns: UI logic is separated from business logic
- Testability: Controllers can be unit tested without Android dependencies
- Maintainability: Changes to UI don't affect business logic
- Reusability: Same class can work with different views

Design Pattern 2: Singleton Pattern

```
1. package ca.iot.innovators.smart.farm.firestore;
 2.
 3. public class FireStoreService {
 4.     private static FireStoreService instance;
 5.     private final UserManager userManager;
 6.
 7.     private FireStoreService() {
 8.         this.userManager = new UserManager();
 9.     }
10.
11.     public static synchronized FireStoreService getInstance() {
12.         if (instance == null) {
13.             instance = new FireStoreService();
14.         }
15.         return instance;
16.     }
17.
18.     public UserManager getUserManager() { return userManager; }
19. }
20.
```

Singleton Usage:

```
 1. // SignupController.java
 2. public class SignupController implements FirebaseAuthManager.AuthCallback {
 3.     private final FireStoreService fireStoreService;
 4.
 5.     public SignupController(SignupView view, Context context) {
 6.         this.view = view;
 7.         this.inputChecker = new InputChecker(context);
 8.         this.authManager = new FirebaseAuthManager(this);
 9.         this.fireStoreService = FireStoreService.getInstance(); // Singleton access
10.     }
11.
12.     @Override
13.     public void onSuccess(FirebaseUser user) {
14.         // Using the single FireStoreService instance
15.         fireStoreService.getUserManager().saveUser(userModel, callback);
16.     }
17. }
18.
```

Explanation:

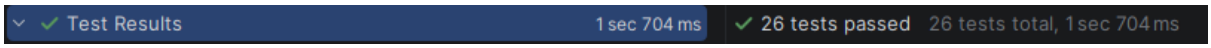
- Single Instance: The FireStoreSerivce creates only one instance of Firestore Service, preventing multiple database connections
- Global Access Point: The getInstance() method make it easy to access the same instance from anywhere in the application

Unit Test Cases

Demonstrate the use of assertEqual, assertTrue, assertFalse, and asssertNotNull in your testing

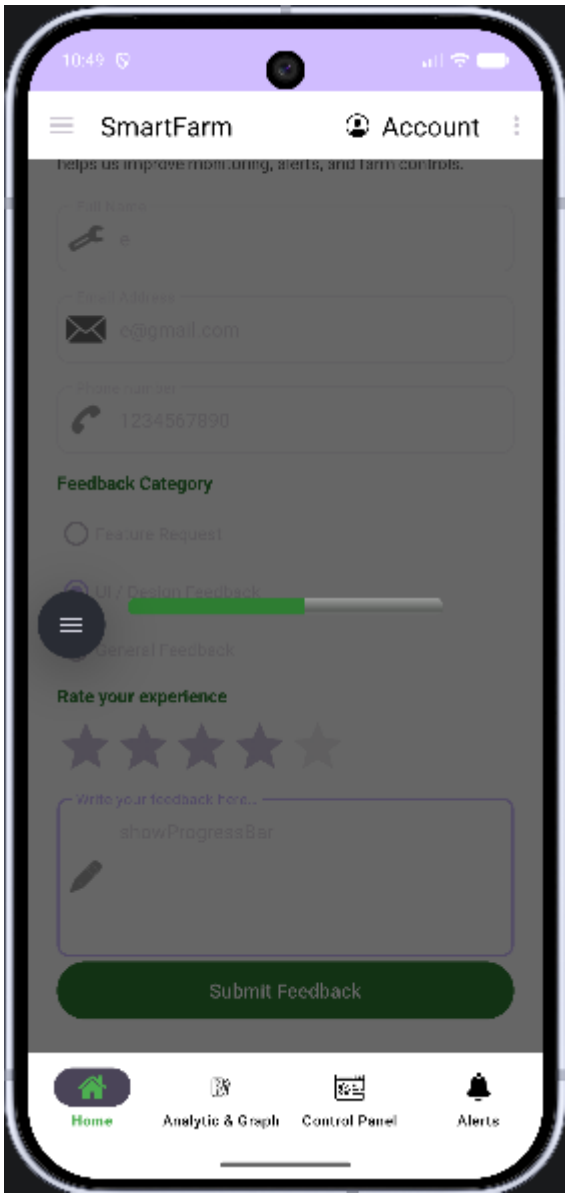
```
 1.     @Test
 2.     public void testValidateLogin_EmptyEmail_Invalid() {
 3.         InputChecker.ValidationResult result = inputChecker.validateLogin(
 4.                 "", "Password123!");
 5.
 6.         assertFalse("Empty email should fail validation", result.isValid());
 7.         assertTrue("Email error should be present",
result.hasError(InputChecker.FIELD_EMAIL));
 8.         assertNotNull("Email errors message expected", result.getEmailError());
 9.         assertEquals("Error message should match",
10.                 context.getString(R.string.email_required), result.getEmailError());
11.     }
12.
```

Screenshot showing all test cases are passing.

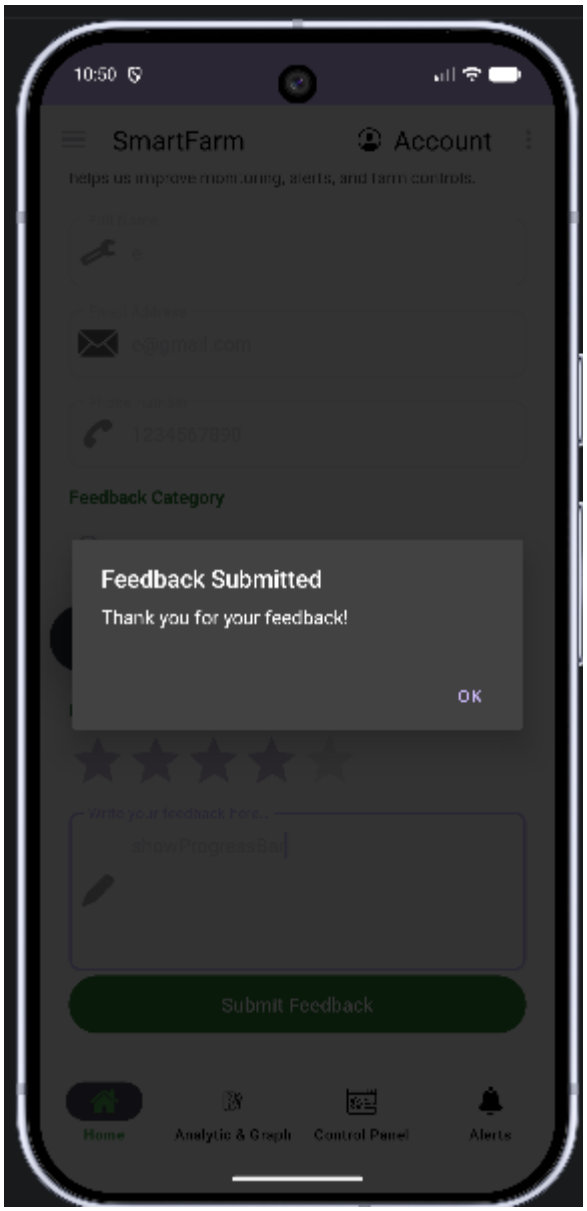


Feedback Screen

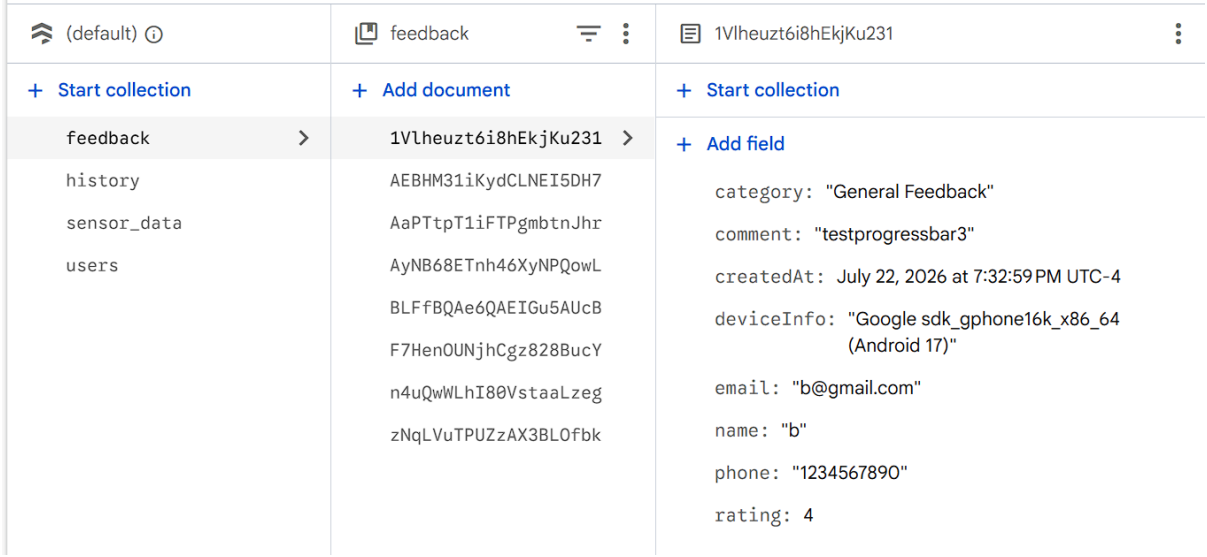
Screenshot showing the progress bar while the form is getting submitted.

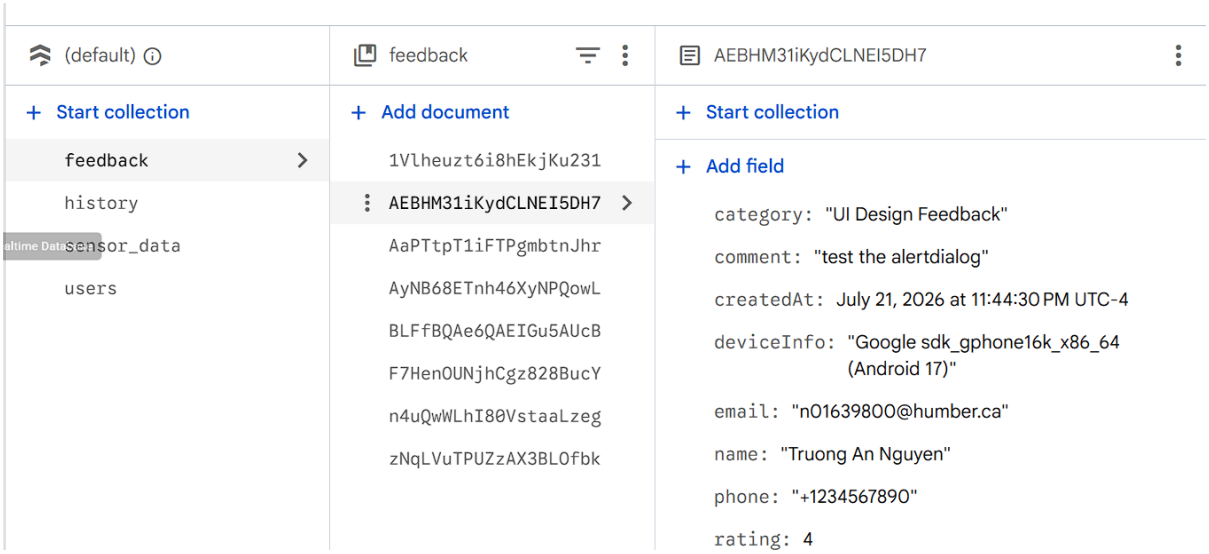


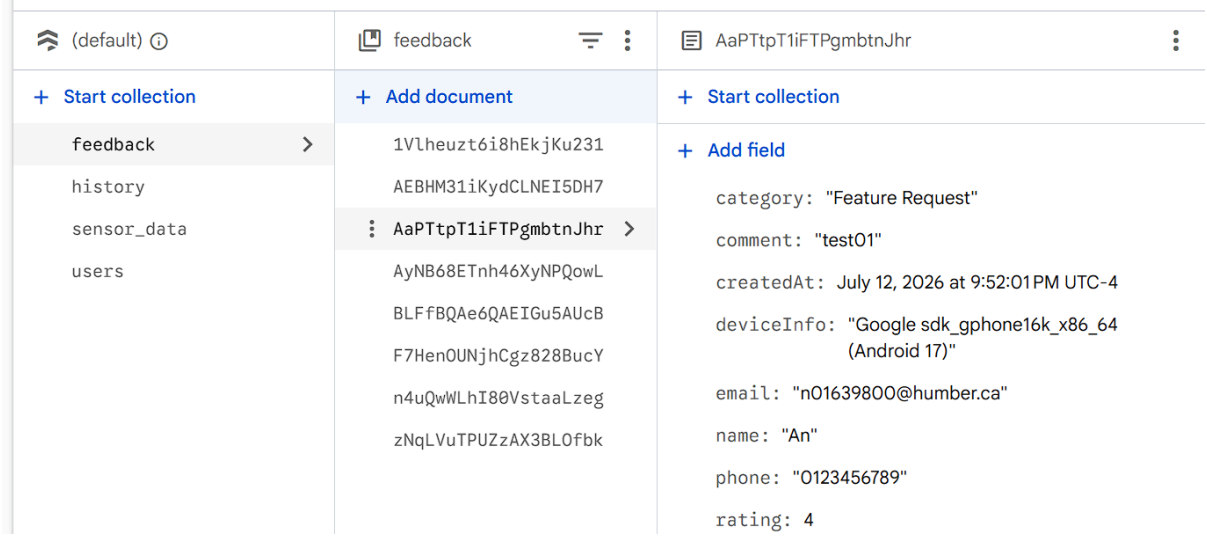
Screenshot showing the AlertDialog once the form is submiled successfully.

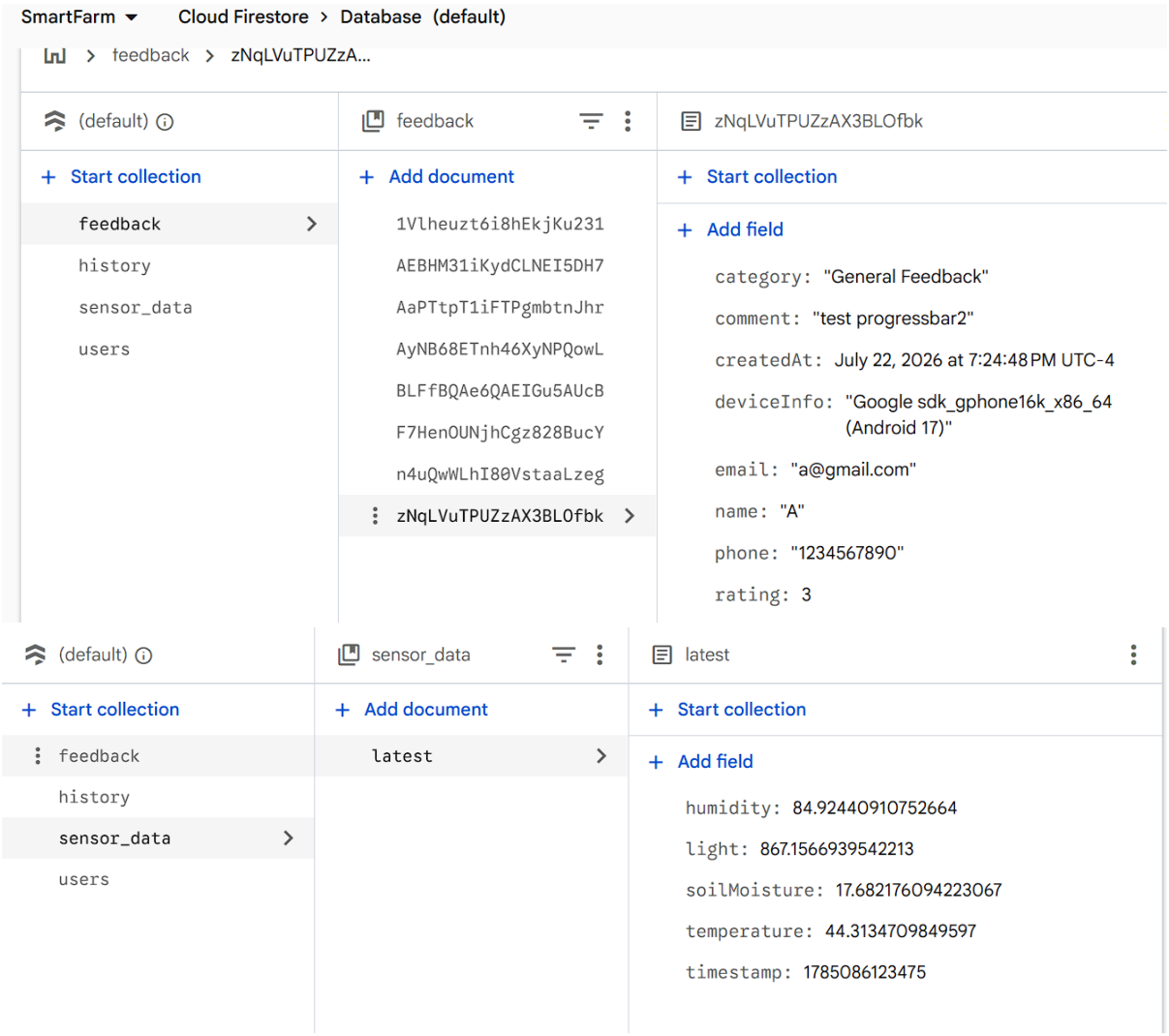


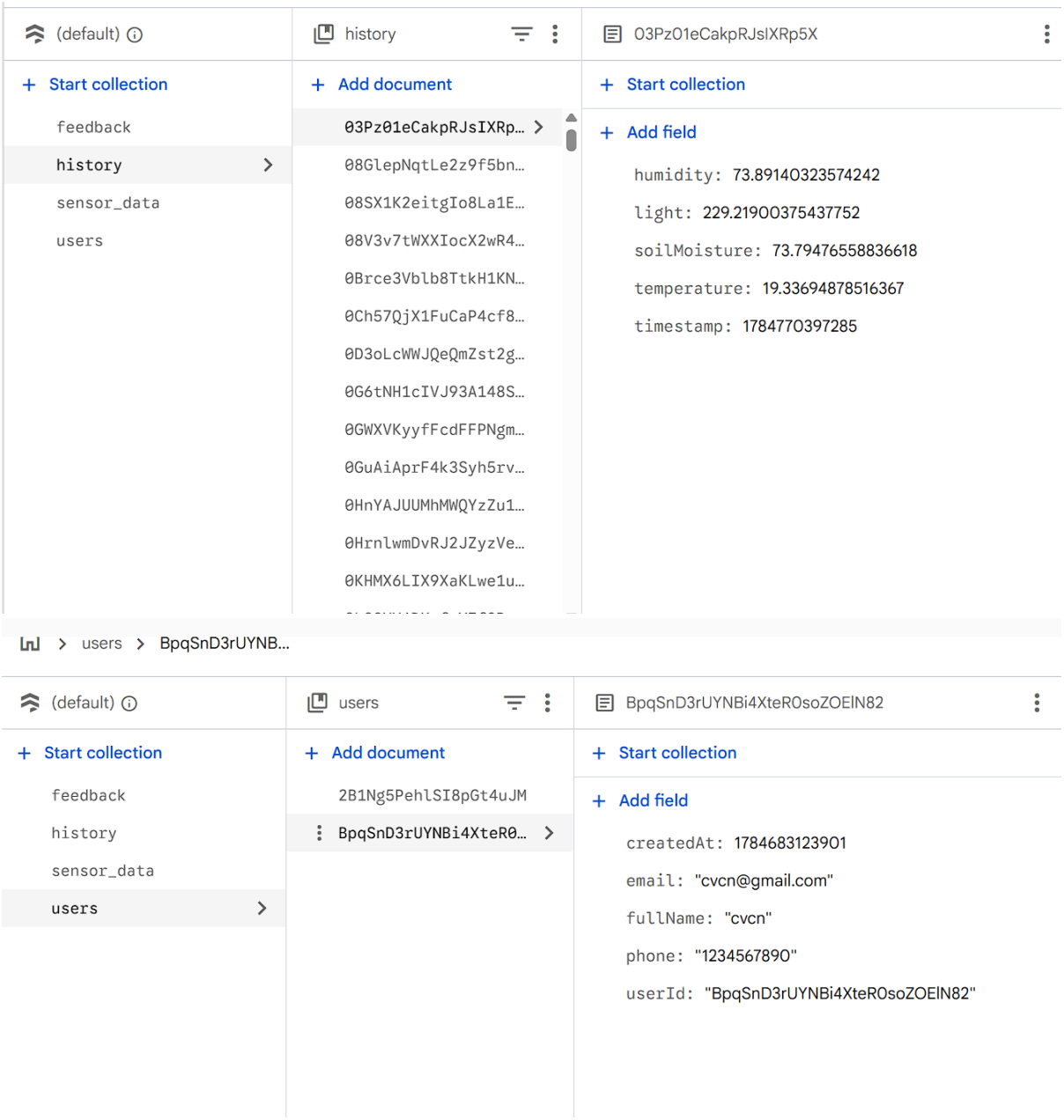
Screenshots showing the data stored into the DB.





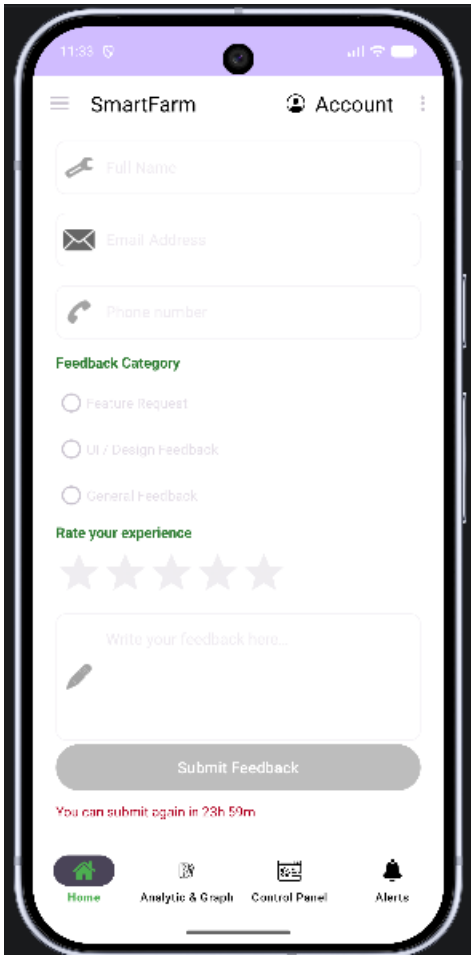



SmartFarm
Cloud Firestore > Database (default)
feedback > zNqLVuTPUZzA...
(default)
feedback
zNqLVuTPUZzAX3BLOfbk
+ Start collection
+ Add document
+ Start collection
feedback
history
sensor_data
users
1Vlheuzt6i8hEkjKu231
AEBHM31iKydCLNEI5DH7
AaPTtpT1iFTPgmbtnJhr
AyNB68ETnh46XyNPQowL
BLFfBQAe6QAEIGu5AUcB
F7HenOUNjhCgz828BucY
n4uQwWLhI80VstaaLzeg
zNqLVuTPUZzAX3BLOfbk
+ Add field
category: "General Feedback"
comment: "test progressbar2"
createdAt: July 22, 2026 at 7:24:48 PM UTC-4
deviceInfo: "Google sdk_gphone16k_x86_64 (Android 17)"
email: "a@gmail.com"
name: "A"
phone: "1234567890"
rating: 3
(default)
sensor_data
latest
+ Start collection
+ Add document
+ Start collection
feedback
history
sensor_data
users
latest
+ Add field
humidity: 84.92440910752664
light: 867.1566939542213
soilMoisture: 17.682176094223067
temperature: 44.3134709849597
timestamp: 1785086123475


(default)
history
03PzO1eCakpRJslXRp5X
+ Start collection
+ Add document
+ Start collection
feedback
history
sensor_data
users
03Pz01eCakpRJsIXRp...
08GlepNqtLe2z9f5bn...
08SX1K2eitgIo8La1E...
08V3v7tWXXIocX2wR4...
0Brce3Vblb8TtkH1KN...
0Ch57QjX1FuCaP4cf8...
0D3oLcWWJQeQmZst2g...
0G6tNH1cIVJ93A148S...
0GWXVKyyfFcdFFPNgm...
0GuAiAprF4k3Syh5rv...
0HnYAJUUMhMWQYzZu1...
0HrnlwmDvRJ2JZyzVe...
0KHMX6LIX9XaKLwe1u...
+ Add field
humidity: 73.89140323574242
light: 229.21900375437752
soilMoisture: 73.79476558836618
temperature: 19.33694878516367
timestamp: 1784770397285
users > BpqSnD3rUYNB...
(default)
users
BpqSnD3rUYNBi4XteROsoZOEIN82
+ Start collection
+ Add document
+ Start collection
feedback
history
sensor_data
users
2B1Ng5PehlSI8pGt4uJM
BpqSnD3rUYNBi4XteR0...
+ Add field
createdAt: 1784683123901
email: "cvcn@gmail.com"
fullName: "cvcn"
phone: "1234567890"
userId: "BpqSnD3rUYNBi4XteROsoZOEIN82"

Clear the user input once the form is submitted. Restrict user submission to once per 24 hrs.

Once the user submits the form successfully, gray out the submit button, and display a timer showing how many hours and minutes remaining when the user can submit another feedback.



Describe how you satisfied this requirement.

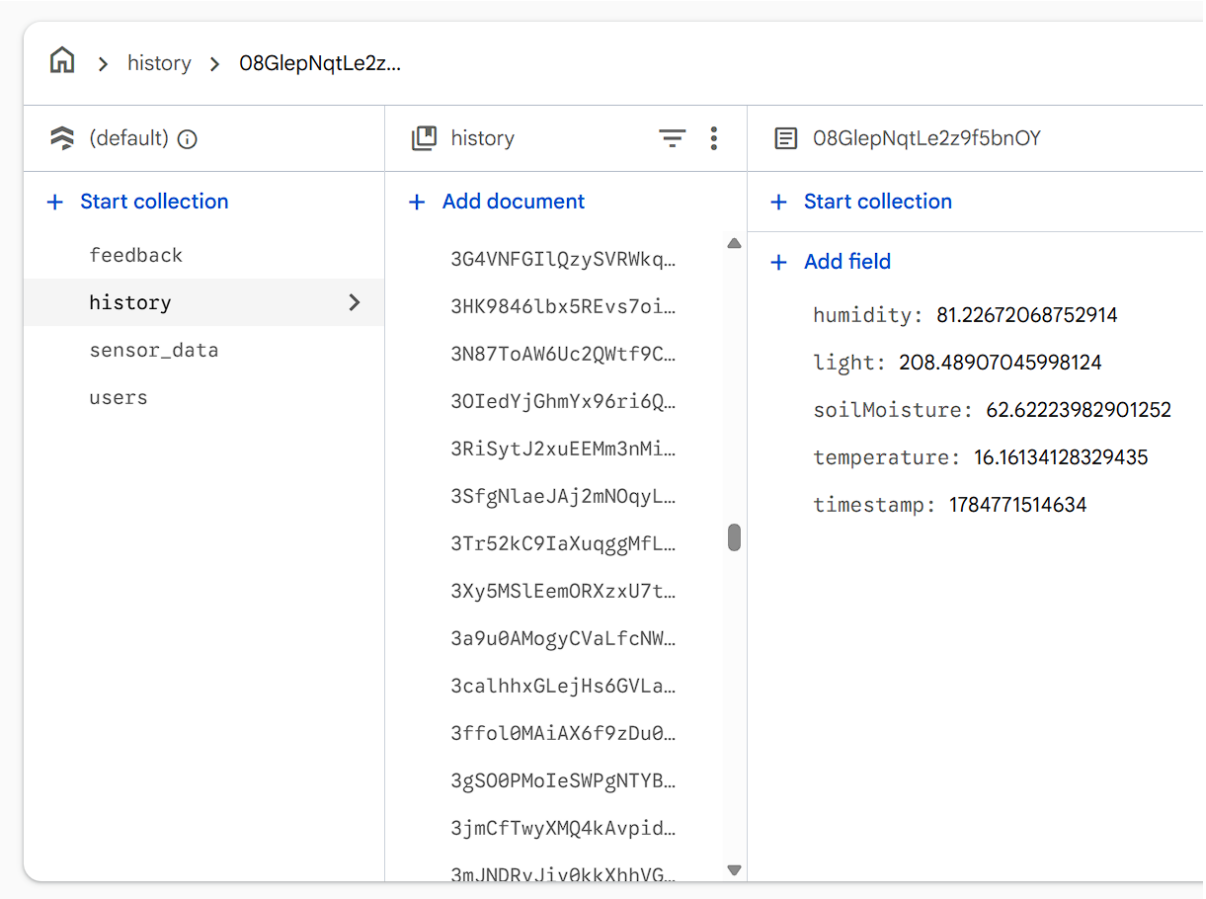
Clearing input on submission: Once the form passes validation, the entered values are saved internally before the input fields, category selection, and rating are immediately reset to empty. This ensures the form appears blank underneath the loading overlay while submission is in progress.

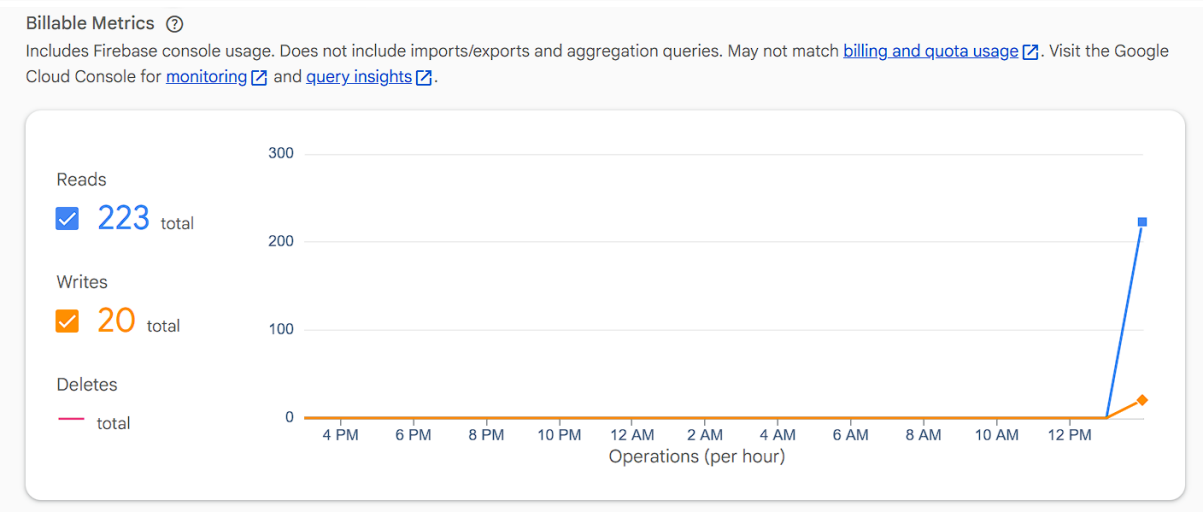
24-hour submission restriction: The timestamp of each user's last successful submission is stored in Firestore under their account ID, rather than on the device. This ensures the restriction follows the user across devices and reinstalls. Before allowing a new submission, the app checks this stored timestamp against the current time to confirm 24 hours have passed.

Graying out the submit button: The submit button uses a color state that automatically switches to gray whenever it is disabled, and green when active. This state is triggered both during the 5-second submission process and throughout the 24-hour cooldown period.

Live countdown display: While the cooldown is active, a timer updates every second, showing the exact hours and minutes remaining before the user can submit again. Once the countdown reaches zero, the button automatically re-enables and the timer disappears, without requiring the user to restart the app.

Take screenshot showing all sensor data fetched and updated from the DB.





Describe the main functionality added in this sprint

1. Real-Time Monitoring (Core Feature)

- Display temperature
- Display humidity
- Display soil moisture
- Display light intensity
- Show device status (online/offline)

2. Device Control

- Turn irrigation pump on/off
- Turn ventilation fan on/off
- Turn grow lights on/off

3. Alerts and Notifications

- High temperature warning
- Low soil moisture warning
- Low water level alert

4. Data History

- View previous sensor readings
- Display charts for temperature, humidity, and soil moisture
- Track environmental changes over time

5. User Authentication

- User login and registration with Firebase Authentication
- Secure access to farm data